A3-PS001-TC171

Android Ransomware-like Activity Early Warning in a Controlled Lab

Susnata Maiti
Team Lead
maitisusnata135531@gmail.com

Moupiya Manna
Author
Moupiya.Manna2024@iem.edu.in

Prof. Baisakhi Das
Author
baisakhi.das@iem.edu.in

AIORI Hackathon 3 • October 5, 2026

Problem Description

Zero-day ransomware
Binary not yet cataloged

→

Signatures miss it
No match, no alert

→

Behaviour remains
Rapid open → write → rename

Problem
Ransomware rewrites and renames many files in seconds. Late detection means lost data.

Approach
Frida hook on file renames in a test directory, scored by a rate rule in a Python engine.

Outcome
3 of 3 simulated attacks stopped, 0 false alarms on 2 benign workloads.

Normal files
Before the attack

photos.jpg | notes.txt | doc.pdf

→ **in seconds**

Locked files
After ransomware-like activity

photos.jpg.enc | notes.txt.enc | doc.pdf.enc

Background

Ransomware file-system pattern

open → write → rename → .enc → repeat fast

Android sandbox
Cannot inspect other apps' memory

⇒

Observe behaviour
Watch file activity in one test directory

Why renames?
Encrypted files are renamed in bulk, a clear signal.

Why a test directory?
Scope stays explicit and safe, with no device-wide claims.

Why an emulator?
Controlled lab with documented telemetry.

> Android blocks one app from reading another's memory, so we watch what is observable: file renames in the test directory.

Literature & RFC Review

Signature AV
Needs a known binary: misses zero-day

Static analysis
Obfuscation can hide intent

Behavioural (UEBA)
Chosen: sees what the app actually does

Approach	Detects zero-day?	Needs known binary?	Speed
Signature AV	No	Yes	Fast
Static analysis	Partly	Yes	Slow
Behavioural (UEBA)	**Yes**	**No**	**Seconds**

Guidance: Android security best practices [1] • Instrumentation: Frida [2]

Three detection approaches compared. Behaviour-based detection suits zero-day threats and needs no real malware.

Proposed Solution



`Red_Team_Simulator.apk`
5 harmless test workloads

→

`ueba_hook.js`
Frida hook on File.renameTo

→

`ueba_engine.py`
⩾ 10 renames in 2 s ⇒ alert

→

`adb force-stop`
Target package only

Workloads in the APK (attackDirectory)

CryptoLocker	DeepTree	PolyWiper	DocAutoSave	PhotoSync
Simulated attack	Simulated attack	Simulated attack	Benign	Benign

How it works

1. Hook sees every rename

2. Engine counts in 2s

3. Kill app, print metrics

> Tap a workload in the APK → Frida hooks its file renames → the Python engine counts them → the app is stopped if the rate is too high.

Optimization Proposed

Aspect	Before	After
Trigger	Any rename	Path / .enc-filtered
Decision	Single event	⩾ 10 in 2 s window
Response	Manual stop	Auto force-stop
Scope	Whole device	One test app

Results: 3 attacks + 2 benign

TP (attacks stopped)
3

FP (benign stopped)
0

FN (attacks missed)
0

TN (benign ignored)
2

Precision 100% • Recall 100%

Threshold = 10
High enough to ignore normal saves.

Window = 2 s
Only recent renames count.

Kill-switch
adb force-stop on the test package only.

> Filtering to test paths and requiring 10 renames in 2 s avoids stopping normal apps. Results: no false alarms.

Solution Architecture & Design



Android emulator

`Red_Team_Simulator.apk`

com.redteam.simulator

`ueba_hook.js`

Frida script in ART VM

⇄

Host PC

`ueba_engine.py`

Window + metrics report

`adb shell`

am force-stop <package>

Frida USB channel: send() → on_message() • logs: [DEBUG] / Rename logged

> The hook runs inside the emulator app. The engine and adb run on the host PC and talk to it over Frida's USB channel.

Data Flow Diagram

1 Simulator
Renames files in test dir

→

2 Hook
Sends source and new path

→

3 Engine
Keeps last 2 s of events

→

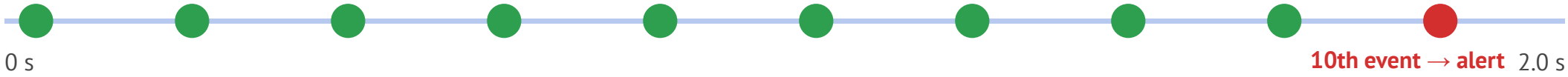
4 Decision
⩾ 10 in 2.0 s?

No
Keep monitoring

Yes
Report delay + files, force-stop

Metrics: measured delay • files changed before alert

Sliding window (2.0 s)



Deployment Timeline

1. Lab setup
Emulator + Frida

→

2. Workloads
3 attacks, 2 benign

→

3. Detector
Hook + engine

→

4. Evaluate
Delay, files, P/R

→

5. Demo & docs
Repo + PDF

Deliverables

Prototype
Frida hook + Python engine

Simulations
APK with 3 attacks, 2 benign

Telemetry
Rename logs and metrics

Evaluation
TP, TN, FP, FN, precision, recall

Five steps from lab setup to demo. Evaluation uses 3 simulated attacks and 2 benign workloads.

Bibliography

[1] Android Developers, “Security best practices”. developer.android.com/privacy-and-security/security-best-practices

[2] Frida, dynamic instrumentation toolkit, documentation. frida.re/docs

Tools used

Android emulator
Controlled test device

Frida
Runtime hooking

adb
Stops the test app

Python
Detection engine



Thank You for Your Attention!

Susnata Maiti • Moupiya Manna • Prof. Baisakhi Das